

CPU Scheduling Simulator

Jonathan Koral

An interactive web application for visualizing and comparing CPU scheduling algorithms

About Me & My Role

Who Am I

- Jonathan Koral — CS Student, Brooklyn College
- Supervised by Prof. Minh Le
- Solo project
- Contact: jon.koral1214@gmail.com

My Responsibilities

- Design the system architecture
- Implement scheduling algorithms in Java
- Build and style the web frontend
- Write and maintain the REST API
- Write JUnit test cases for every algorithm
- Manage schedule & documentation

My Role

Solo Developer

I own every aspect of this project, the Spring Boot backend and REST API, the JavaScript frontend and CSS themes, the JUnit test suite, and deployment to Render.

Problem Statement

The Problem

Students studying CPU scheduling have no way to verify whether their Gantt charts are correct. Textbooks, class notes, and online videos are all static and none can check a student's specific input.

The Solution

An interactive web simulator that accepts any set of processes, runs the selected scheduling algorithm, and instantly returns a Gantt chart, process metrics, and summary statistics.

Students

Verify Gantt charts and check homework answers

Instructors

Present live classroom demonstrations

CS Learners

Build intuition for preemptive vs non-preemptive scheduling

Value of This Project

Instant Personalized feedback

No more guessing if a Gantt chart is correct — results are generated live for any input.

5 Algorithms supported

FCFS, SJF, SRTF, Round Robin, and Priority — all runnable and comparable side-by-side.

68 JUnit tests written

Covers correctness, edge cases, and boundary values for every algorithm.

0 Hardcoded results

Every output is computed live by the backend. Nothing is pre-calculated on the frontend.

Computer Science Concepts Applied

Algorithms & Complexity

- FCFS: $O(n \log n)$
- SJF: $O(n^2 \log n)$
- SRTF: $O(n^2)$
- RR: $O(n^2)$
- Priority: $O(n^2 \log n)$

Operating Systems

CPU scheduling, Gantt charts, process states, waiting time, turnaround time, and throughput

Software Engineering

Strategy design pattern, REST API design, MVC separation (Controller / Service / Model)

Data Structures

ArrayDeque for ready queues, HashMap for algorithm lookup, ArrayList for Gantt entries

Software Testing

JUnit 5 with 68 unit tests covering boundary values, edge cases, and algorithm correctness

Web Technologies

Java Spring Boot backend communicates with an ES-module JavaScript frontend via JSON over HTTP

Tools & Technologies

Backend

- Java 21
- Spring Boot 4.0.3
- Spring Web (REST)
- Lombok
- Maven

Frontend

- HTML / CSS / JavaScript
- ES Modules (7 JS files)
- Fetch API
- CSS Custom Properties
- 3-theme system

Testing

- JUnit 5
- Postman (API testing)
- Browser DevTools

Dev Tools

- IntelliJ IDEA
- GitHub
- Render (deployment)

Project Timeline — All Phases Complete

Weeks 1–4



- Project setup & Spring Boot scaffold
- FCFS algorithm + JUnit tests
- Basic HTML/CSS frontend
- Initial wireframe & design

Weeks 5–8



- SJF & SRTF algorithms + tests
- Gantt chart renderer
- Modal + process card UI
- Deployed to Render

Weeks 9–12



- Round Robin & Priority + tests
- Algorithm comparison feature
- ES module frontend refactor
- Compare mode UI

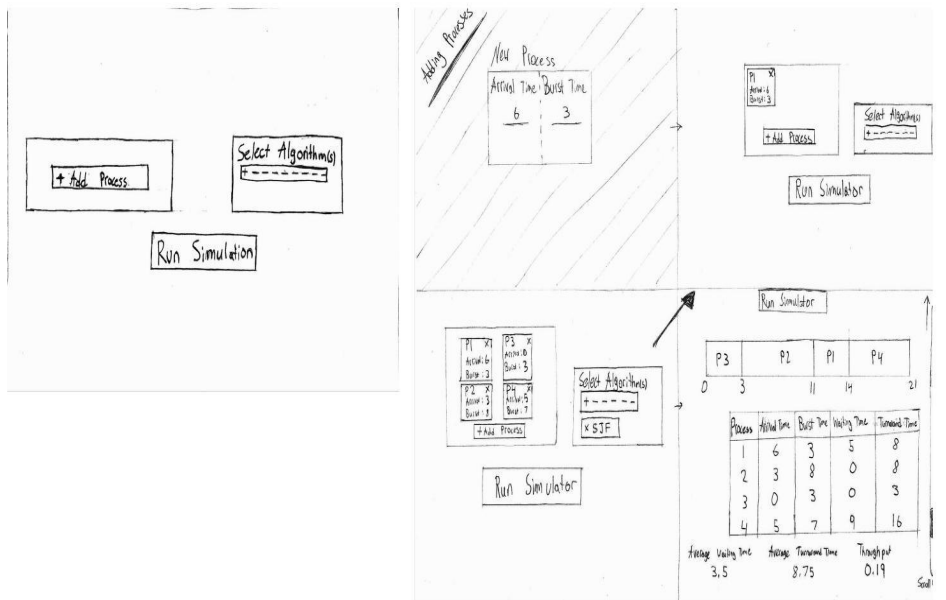
Weeks 13–15



- Theme system (Light/Dark/Blue)
- Random process generation
- Recent Runs sidebar + replay
- Final UI polish & docs

Wireframe — Initial Design

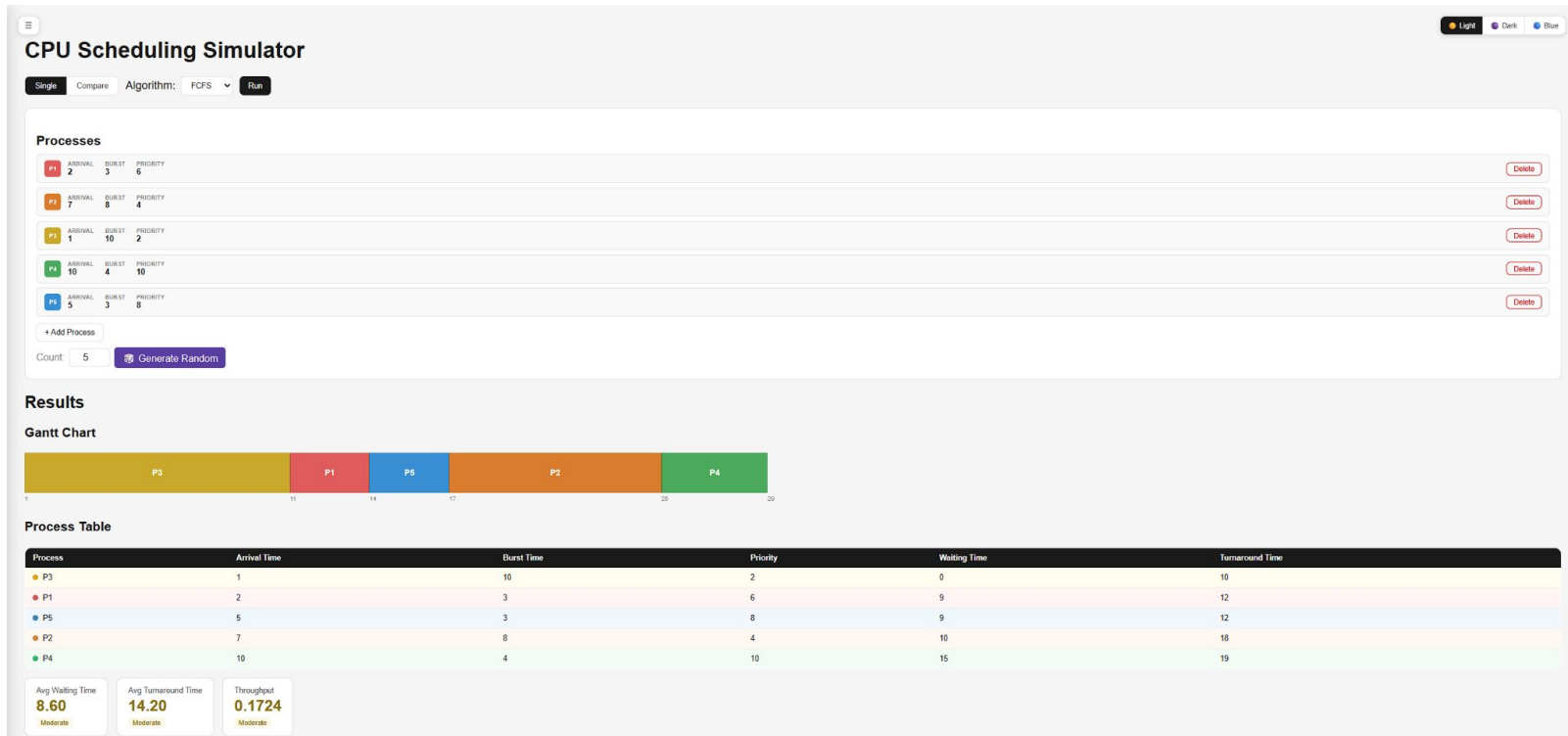
Wireframe



What Changed

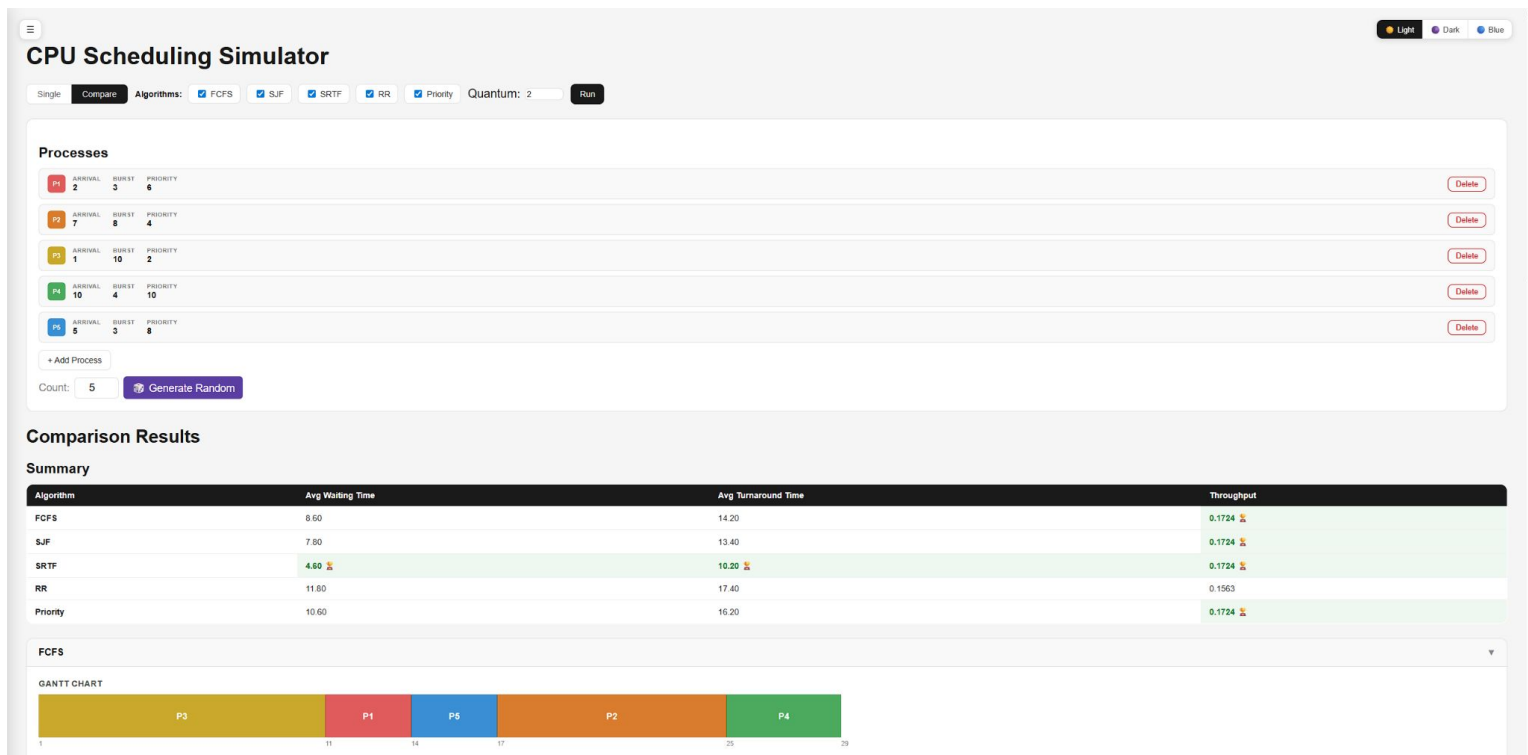
- "Select Algorithm" moved above "Add Process." Run button placed inline.
- Modal redesigned from horizontal to vertical. Priority field added as optional input.
- Gantt chart gained Idle blocks to show CPU idle gaps.
- Compare mode added which was not in the original wireframe.
- Theme switcher and Recent Runs sidebar added as final touches.

Current Version — Single Mode



All output is generated live by the Spring Boot backend. Nothing is hardcoded on the frontend.

Current Version — Compare Mode



Compare mode runs multiple algorithms on the same process set and shows a side-by-side summary with a trophy icon on the best value per metric.

Current Version — Compare Mode



Compare mode runs multiple algorithms on the same process set and shows a side-by-side summary with a trophy icon on the best value per metric.

Current Version — Compare Mode



Compare mode runs multiple algorithms on the same process set and shows a side-by-side summary with a trophy icon on the best value per metric.

UI/UX Design Choices

Theme System

Light, Classic Dark, and Dark Blue themes toggle instantly. Preference saved to localStorage and persists across sessions.

Gantt Tooltips

Hovering any Gantt block shows exact run times, duration, and wait time guides the user to read the chart correctly.

Stat Cards with Tiers

Avg Wait, Turnaround, and Throughput cards turn green, amber, or red by threshold — no manual interpretation needed.

Compare Winner Highlight

The best value per column gets a green highlight and trophy emoji, making the best algorithm for the processes obvious.

Recent Runs Sidebar

Hamburger button (☰) opens a sidebar with up to 20 saved runs. One-click replay restores processes and re-runs the simulation.

Random Process Generator

One click generates 5 processes with randomized values including priority, removing manual input for testing and exploration.

Features Added — Final Phase

Random Process Generation

A count input and Generate Random button creates 5 processes instantly with random arrival (0–14), burst (1–10), and priority (1–10) values. Priority is always generated with no more zero defaults.

Three-Theme System

CSS custom properties in a single theme.css file drive Light, Classic Dark, and Dark Blue themes across all 8 CSS files. Theme choice persists in localStorage.

Recent Runs Sidebar

A hamburger-triggered sidebar saves up to 20 runs with algorithm, process count, stats, and a relative timestamp. Works for both Single and Compare modes.

One-Click Replay

Clicking any recent entry fully restores the saved processes, sets the algorithm selector or compare checkboxes, and automatically re-runs the simulation.

Theme System — Three Modes

☰

LightDarkBlue

CPU Scheduling Simulator

SingleCompareAlgorithm: FCFSRun

Processes

P1	ARRIVAL 14	BURST 2	PRIORITY 5	Delete
P2	ARRIVAL 9	BURST 9	PRIORITY 2	Delete
P3	ARRIVAL 1	BURST 7	PRIORITY 1	Delete
P4	ARRIVAL 14	BURST 5	PRIORITY 3	Delete
P5	ARRIVAL 1	BURST 8	PRIORITY 4	Delete

+ Add Process

Count: 5Generate Random

☰

LightDarkBlue

CPU Scheduling Simulator

SingleCompareAlgorithm: FCFSRun

Processes

P1	ARRIVAL 14	BURST 2	PRIORITY 5	Delete
P2	ARRIVAL 9	BURST 9	PRIORITY 2	Delete
P3	ARRIVAL 1	BURST 7	PRIORITY 1	Delete
P4	ARRIVAL 14	BURST 5	PRIORITY 3	Delete
P5	ARRIVAL 1	BURST 8	PRIORITY 4	Delete

+ Add Process

Count: 5Generate Random

☰

LightDarkBlue

CPU Scheduling Simulator

SingleCompareAlgorithm: FCFSRun

Processes

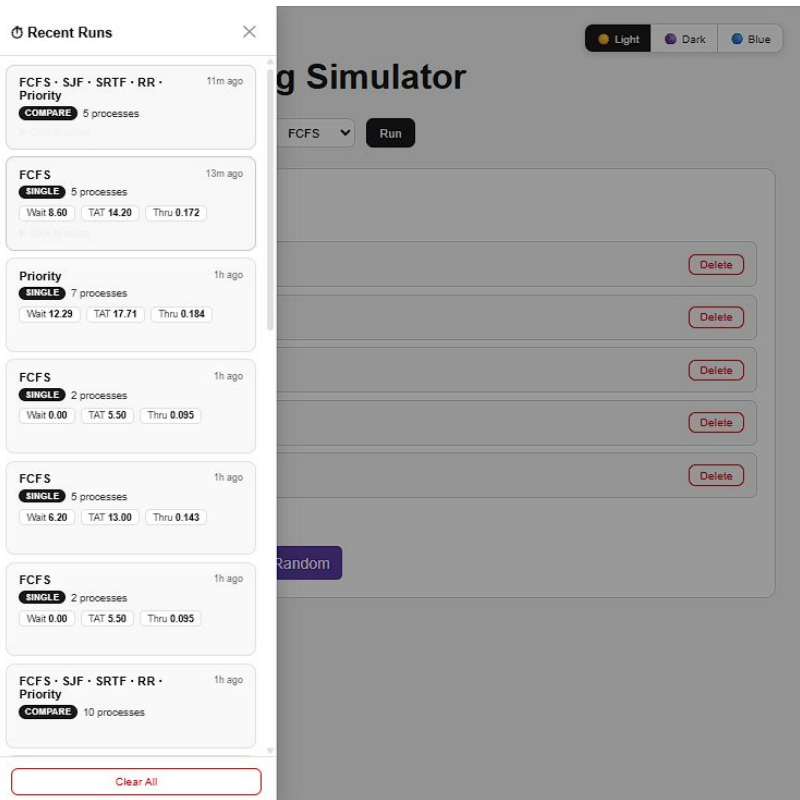
P1	ARRIVAL 14	BURST 2	PRIORITY 5	Delete
P2	ARRIVAL 9	BURST 9	PRIORITY 2	Delete
P3	ARRIVAL 1	BURST 7	PRIORITY 1	Delete
P4	ARRIVAL 14	BURST 5	PRIORITY 3	Delete
P5	ARRIVAL 1	BURST 8	PRIORITY 4	Delete

+ Add Process

Count: 5Generate Random

We have three different themes with the help of CSS custom properties defined in one theme.css file. My Personal favorite is the dark blue theme.

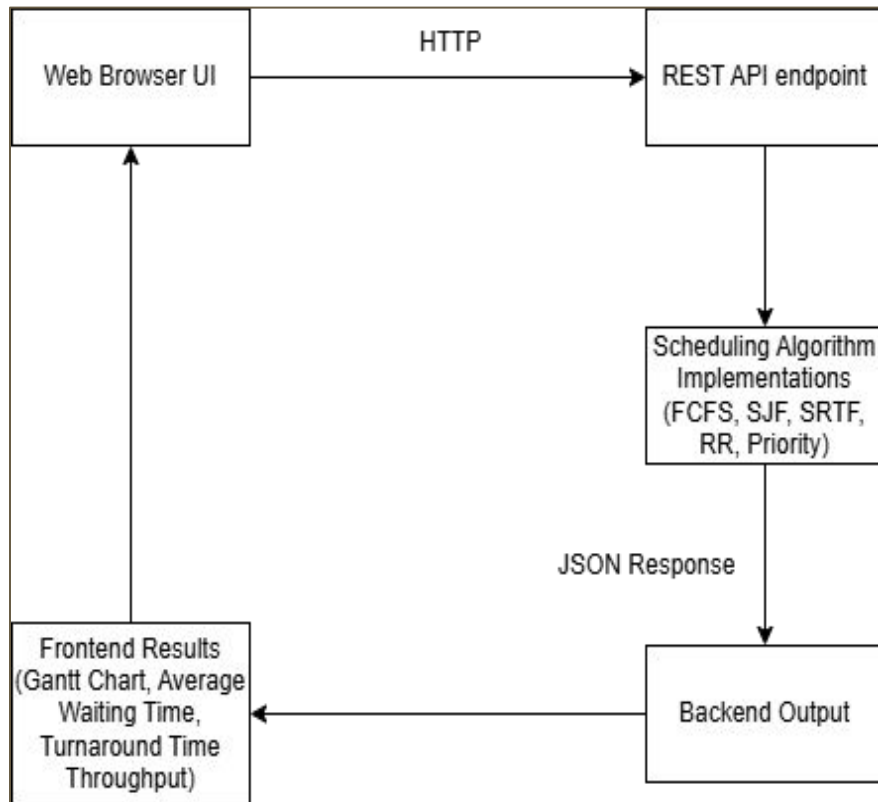
Recent Runs Sidebar



How It Works

- Opened by the ☰ hamburger button fixed to the top-left corner.
- Every successful run is automatically saved: algorithm, mode, process count, stats, and timestamp.
- Timestamps show as relative time: "Just now", "5m ago", "2h ago".
- Clicking any entry restores the saved processes, sets the algorithm selector, and re-runs the simulation automatically.
- Stores up to 20 entries in localStorage

System Architecture



Key Components

Web Browser UI

HTML/CSS/JS — sends JSON POST to backend

SimulationController

POST /api/simulate and
POST /api/compare

SchedulerService

HashMap lookup to select correct algorithm

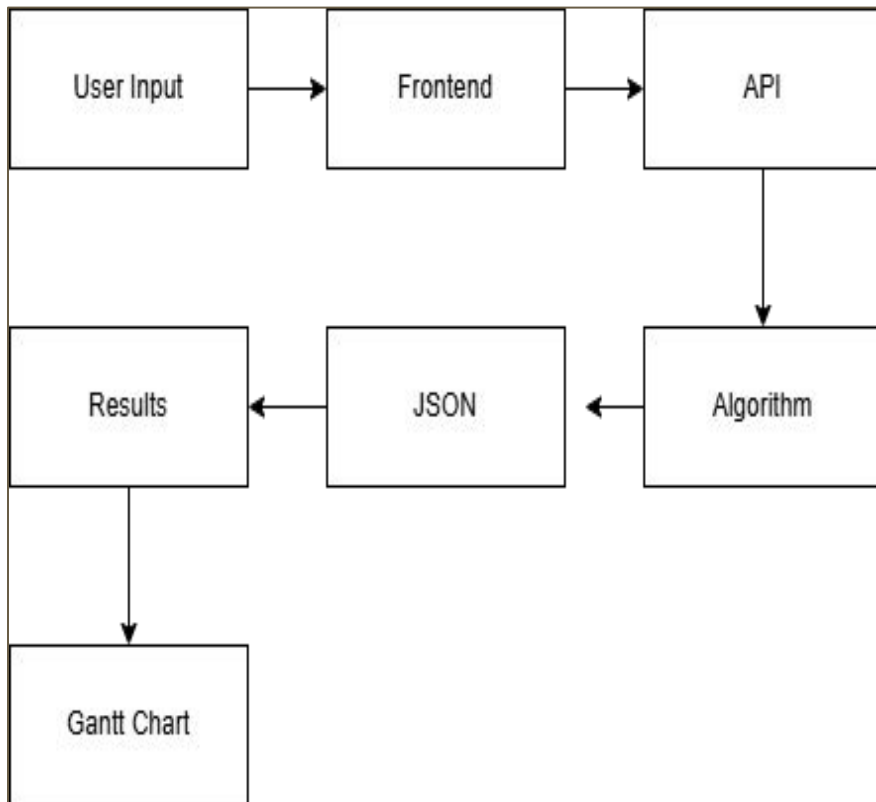
AlgorithmStrategy

Interface: FCFS, SJF, SRTF, RR, Priority

ScheduleResult

DTO: Gantt, process table, averages, throughput

Data Flow Diagram



Data at Each Stage

User Input	processId, arrivalTime, burstTime, priority, algorithm
Frontend	Validates fields, builds JSON request body
API Request	ScheduleRequest sent as JSON to backend
Algorithm	Takes process list, returns ScheduleResult
JSON Response	ganttChart, processTable, averages, throughput
Results UI	Renders Gantt chart, table, and stat cards

REST API Design

POST **/api/simulate**

Single algorithm simulation

Request: { algorithm, quantum?, processes[] }

Response: { ganttChart, processTable, avgWaitingTime, avgTurnaroundTime, throughput }

POST **/api/compare**

Multi-algorithm comparison

Request: { algorithms[], quantum?, processes[] }

Response: { results: { "FCFS": ScheduleResult, "SJF": ScheduleResult, ... } }

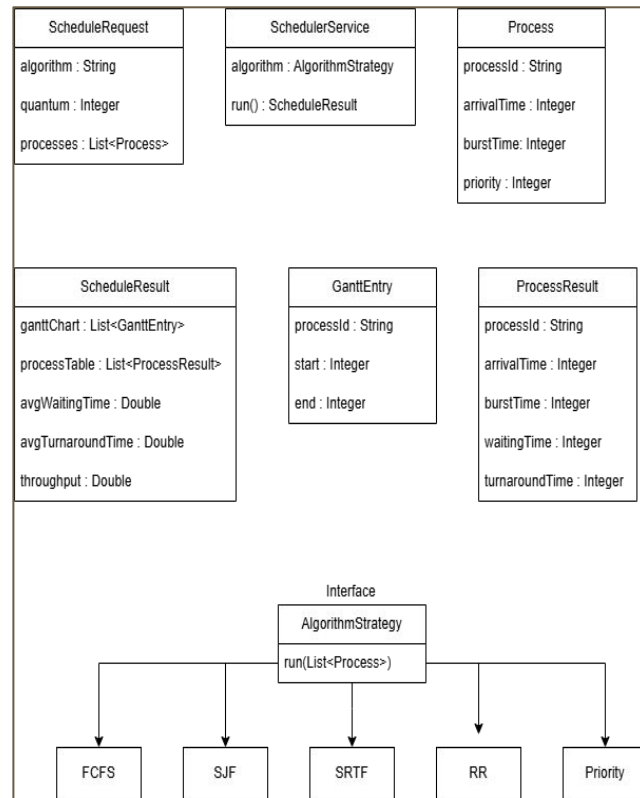
Design Pattern — Strategy Pattern

Why Strategy Pattern?

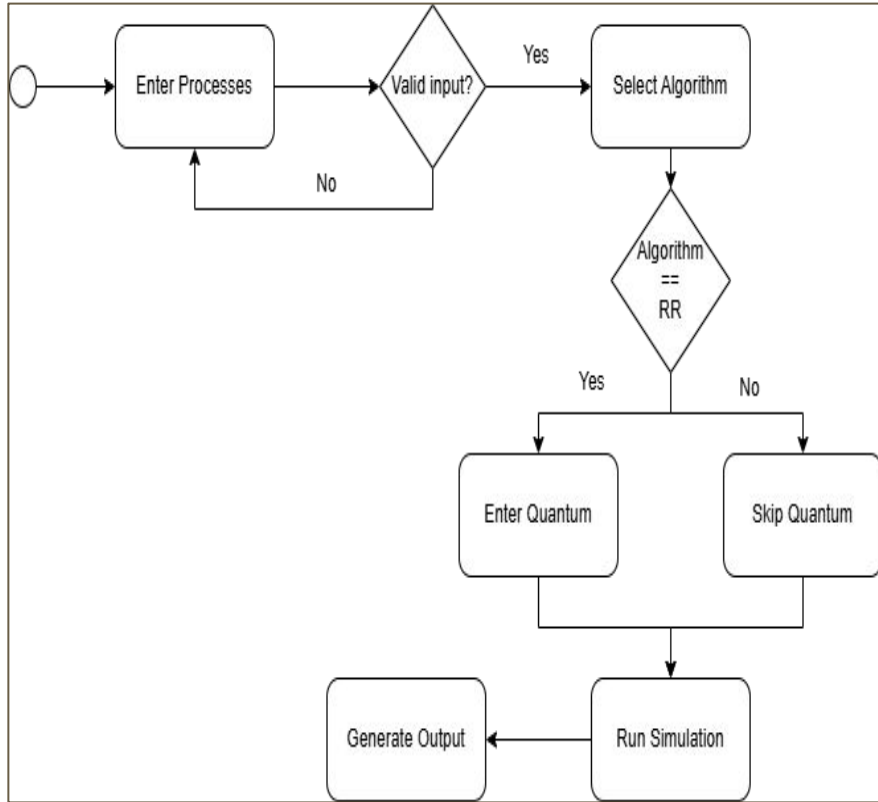
- Each algorithm implements a shared AlgorithmStrategy interface.
- SchedulerService holds a HashMap<String, AlgorithmStrategy> and selects the right one at runtime.
- Adding Priority scheduling required one new class with zero changes to existing code.
- Open/Closed Principle: open for extension, closed for modification.

Key Classes

- AlgorithmStrategy (interface)
- FCFSAlgorithm, SJFAlgorithm, SRTFAlgorithm, RRAAlgorithm, PriorityAlgorithm
- SchedulerService, SimulationController
- Process, GanttEntry, ProcessResult, ScheduleRequest, ScheduleResult



Activity Diagram



Activity Highlights

- User enters processes and selects algorithm.
- If Round Robin is selected, quantum input appears.
- Frontend validates all fields before sending to backend.
- Backend selects the correct algorithm via HashMap.
- Algorithm runs and packages results into ScheduleResult.
- Frontend renders Gantt chart, process table, and stat cards.

Scheduling Algorithms — FCFS & SJF

FCFS

$O(n \log n)$

Non-preemptive

Processes run in arrival order. The only expensive step is the initial sort by arrival time: $O(n \log n)$. After that, a single $O(n)$ pass processes each job. Simple but can produce long wait times if a large job arrives first (convoy effect).

SJF

$O(n^2 \log n)$

Non-preemptive

Every time the CPU becomes free, the ready queue is sorted by burst time to find the shortest job. This sort happens up to n times, each costing $O(n \log n)$, giving $O(n^2 \log n)$ total. Minimizes average wait time but requires knowing burst times in advance.

Scheduling Algorithm — SRTF

SRTF

$O(n^2)$

Preemptive

At every clock tick, the entire ready queue is scanned for the process with the least remaining burst time which is $O(n)$ per tick. The simulation runs for $O(n)$ ticks in the worst case, giving $O(n) \times O(n) = O(n^2)$ total.

Extra Tracking Needed

- Remaining burst time per process (HashMap)
- Current process ID to detect preemptions
- Tie-break by process ID when burst times are equal

Unique Challenge

SRTF is the most complex algorithm to simulate because a running process can be interrupted at any clock tick. The Gantt chart must correctly track every preemption and resumption as separate blocks.

Key Design Decision

A new GanttEntry must be created whenever the running process changes, not just when a process finishes.

Scheduling Algorithms — RR & Priority

Round Robin

$O(n^2)$

Preemptive

Each process gets a fixed time quantum. If it doesn't finish, it goes back to the end of the queue. Requires a circular queue (ArrayDeque) and remaining burst tracking. Fair but can produce high turnaround times with many short processes.

Priority

$O(n^2 \log n)$

Non-preemptive

Ready queue is sorted by priority number (lower number = higher priority) each time the CPU becomes free. Ties are broken by arrival time. Built on the same non-preemptive structure as SJF but sorted by priority instead of burst time.

Algorithm Comparison Feature

Processes

P11	ARRIVAL 4	BURST 4	PRIORITY 9	Delete
P12	ARRIVAL 8	BURST 5	PRIORITY 10	Delete
P13	ARRIVAL 3	BURST 8	PRIORITY 1	Delete
P14	ARRIVAL 10	BURST 1	PRIORITY 8	Delete
P15	ARRIVAL 13	BURST 4	PRIORITY 2	Delete

+ Add Process

Count: 5

Generate Random

Comparison Results

Summary

Algorithm	Avg Waiting Time	Avg Turnaround Time	Throughput
FCFS	6.40	10.80	0.2000 🏆
SJF	4.80	9.20	0.2000 🏆
SRTF	3.20 🏆	7.60 🏆	0.2000 🏆
RR	6.20	10.60	0.1923
Priority	4.80	9.20	0.2000 🏆

How It Works

- User selects multiple algorithms via checkboxes.
- One POST /api/compare request with all selected algorithms.
- Backend deep-copies the process list for each algorithm to prevent mutation.
- Results returned as a LinkedHashMap preserving order.
- Best value per metric highlighted green with 🏆.

Toughest Bug — SRTF Gantt Chart Splitting

The Bug

SRTF is preemptive — a process can be interrupted and resume later. My initial implementation merged interrupted and resumed segments into one block, making the chart wrong.

Expected output

[P1: 0-3] [P2: 3-5] [P1: 5-9]

Actual output (bug)

[P1: 0-9]

The fix: a Gantt entry must be created whenever the running process changes, not just when a process finishes.

The Fix

```
if (currentId.equals("") ||
    !currentId.equals(
        shortest.getProcessId())) {

    if (!currentId.equals("")) {
        // Add entry on switch,
        // not just on finish
        ganttChart.add(
            new GanttEntry(
                currentId, curr, end
            ));
    }
    currentId =
        shortest.getProcessId();
    curr = end;
}
```

Frontend Refactor — ES Modules

Before

- 1 large main.js with all logic mixed together
- 1 large styles.css
- Inline onclick handlers in HTML
- No separation of concerns

After

- 7 specific JS modules
- 8 scoped CSS files
- All event listeners in main.js
- No build tools needed because Spring Boot serves static files over HTTP

Module Breakdown

state.js	Shared processes array, colors, counters
modal.js	Add/delete process, random generation
mode.js	Single/Compare toggle, quantum visibility
api.js	All fetch calls to the backend
render.js	Gantt builder, process table, stat cards
compare.js	Comparison table, collapsible sections
main.js	Wires all event listeners on DOM ready

Testing — JUnit 5 Test Suite

68

Total Tests

5

Algorithms Covered

100%

Pass Rate

0

Hardcoded Assertions

Test Categories

- Correctness: Gantt order, start/end times
- Per-process waiting & turnaround time
- Average waiting time & turnaround time
- Throughput = $n / \text{last end time}$

Edge Cases

- Single process at $t=0$ and arriving late
- All processes at same non-zero time
- Max burst (99) and max arrival (99)
- SRTF: preemption + Gantt splitting
- SJF: tie-break by arrival time
- Priority 0 treated as highest priority

Use Case 1 — Student Checks Homework

Scenario

A student uses the simulator to verify Gantt charts before submitting homework.

Processes Entered

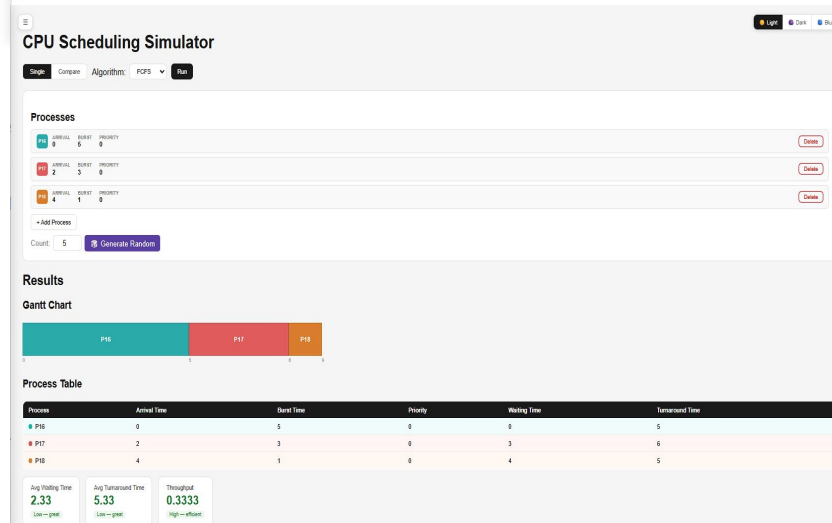
- P1: arrival 0, burst 5
- P2: arrival 2, burst 3
- P3: arrival 4, burst 1

Value

No more guessing. Personalized, instant feedback for any input.

Output

Gantt chart with exact block timings, per-process waiting and turnaround times, averages, and throughput — instantly confirming or correcting the student's handwritten answer.



Use Case 2 — Comparing Algorithms

Scenario

A user wants to understand how different algorithms behave on the same workload.

Processes Entered

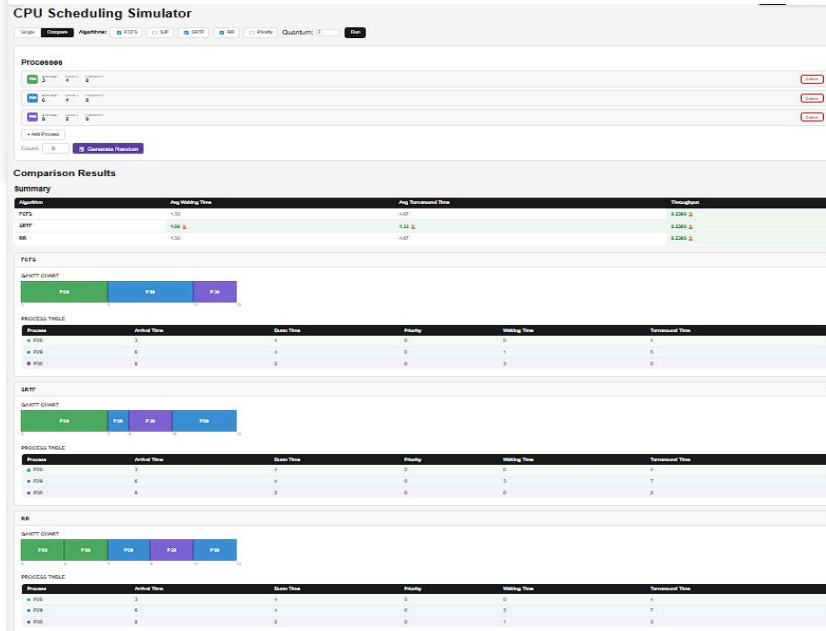
- P1: arrival 3, burst 4
- P2: arrival 6, burst 3
- P3: arrival 8, burst 2

Value

Makes algorithm tradeoffs visible and concrete instead of theoretical.

Output

Side-by-side summary for FCFS, SRTF, and RR (Q=2).
Best value per metric highlighted green with a trophy.
Full Gantt charts available in collapsible sections below.



Use Case 3 — Instructor Demonstration

Scenario

An instructor demonstrates Priority scheduling to the class in real time.

Processes Entered

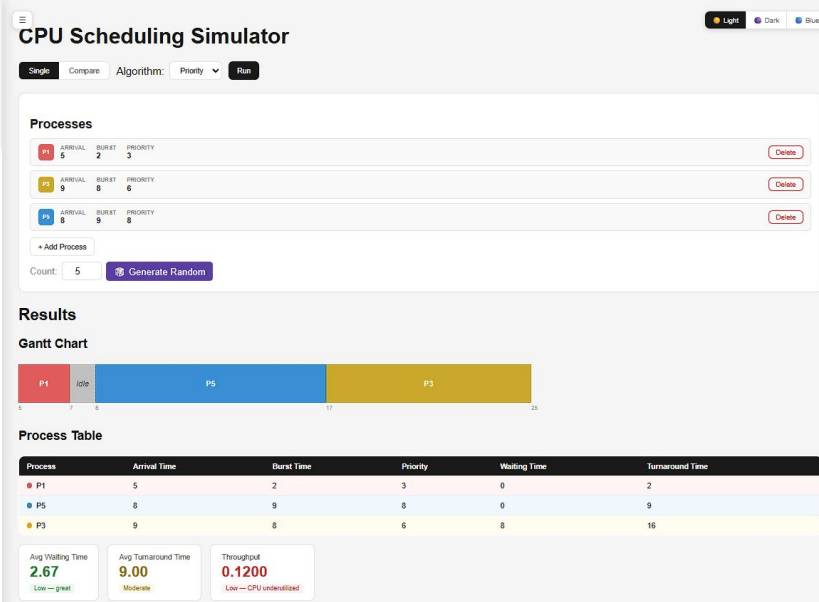
- P1: arrival 5, burst 2, priority 3
- P2: arrival 9, burst 8, priority 6
- P3: arrival 8, burst 9, priority 8

Value

A live, interactive tool that responds to any input the instructor chooses.

Output

Gantt chart shows $P2 \rightarrow P3 \rightarrow P1$, demonstrating that lower priority number = higher priority. Waiting times and turnaround visible immediately for class discussion.



Scope Changes & Feature Decisions

Not Pursued

- User accounts & saved simulations which requires a database.
- Animated Gantt chart with real-time step-through was thought of but cut to keep the backend-driven architecture clean.
- Mobile-responsive layout but focus stayed on desktop. Not optimized for small screens.

Completed & Most Excited About

- Algorithm comparison mode which can run all 5 algorithms on the same data in one click with a trophy-highlighted summary was the most satisfying feature to ship.
- Recent Runs with replay. One-click replay turned the simulator into something you can genuinely learn from iteratively.
- Three-theme system. This is a fully theme-aware UI across 8 CSS files.

Reflection & What's Next

What I Learned

- How to design a clean REST API and consume it from a vanilla JS frontend without any framework.
- How the Strategy pattern makes a codebase genuinely more better. Adding more algorithms touched zero existing files.
- How CSS custom properties can create a full multi-theme system elegantly.
- How to structure a real JUnit test suite with useful edge cases.

If I Continued

- Animated Gantt chart with step-through playback showing each time unit executing.
- UI fix especially for use on tablets and phones.
- Export results as PDF or CSV for students to submit with homework.
- Multilevel Feedback Queue (MFQ) as a sixth scheduling algorithm.

Project Links

GitHub Repository

<https://github.com/J-Koral/cpu-scheduling-simulator.git>

Live Website

<https://j-koral.github.io/cpu-scheduling-simulator/>

Project Management Board

<https://github.com/users/J-Koral/projects/1>

Software Documentation

<https://github.com/J-Koral/cpu-scheduling-simulator/tree/master/src/main/resources/SoftwareDocumentation>